

Product Management API Documentation

AI-Powered Multi-Branch Retail POS & Inventory Management System

This document explains the Product Management backend APIs in a simple way, with endpoint examples and Postman usage steps. The module covers product CRUD operations, image upload, barcode search, search/filter, activation/deactivation, and permanent delete.

1. Base URL

`http://localhost:5000/api/products`

2. Product Fields

A product document stores the main product details. The actual image is stored in Cloudinary, and only the image URL is saved in MongoDB.

```
{
  "name": "Coca Cola",
  "barcode": "123456789111",
  "brand": "Coca Cola",
  "description": "Soft drink",
  "price": 250,
  "costPrice": 180,
  "image": "https://res.cloudinary.com/...",
  "reorderLevel": 20,
  "unit": "bottle",
  "isActive": true
}
```

3. Simple System Flow

Add Product Flow: Admin enters product details -> Admin selects image -> Backend uploads image to Cloudinary -> Cloudinary returns image URL -> Backend saves product details and image URL in MongoDB -> Product appears in product list.

Barcode Flow: Cashier scans or enters barcode -> System sends barcode to backend -> Backend searches product by exact barcode -> Product details return to POS screen -> Product can be added to bill.

Soft Delete Flow: Admin deactivates product -> Product `isActive` becomes false -> Product remains in MongoDB -> Product can be reactivated later.

Real Delete Flow: Admin deletes product -> Product is permanently removed from MongoDB.

4. API Endpoints

4.1. Add Product with Image

Purpose: Adds a new product. The image uploads to Cloudinary and the image URL is saved in MongoDB.

Method	POST
Endpoint	/api/products
Full URL Example	http://localhost:5000/api/products

How to use in Postman: Select Body -> form-data. Add text fields and set image field type to File.

Key	Type	Example Value
name	Text	Coca Cola
barcode	Text	123456789111
brand	Text	Coca Cola
description	Text	Soft drink
price	Text	250
costPrice	Text	180
reorderLevel	Text	20
unit	Text	bottle
image	File	Choose image file

Example Response:

```
{
  "success": true,
  "message": "Product added successfully",
  "product": {
    "_id": "PRODUCT_ID",
    "name": "Coca Cola",
    "barcode": "123456789111",
    "price": 250,
    "image": "https://res.cloudinary.com/..."
  }
}
```

4.2. Get All Products

Purpose: Returns all products from the database.

Method	GET
Endpoint	/api/products
Full URL Example	http://localhost:5000/api/products

How to use in Postman: No body required. Send a GET request.

Example Response:

```
{
  "success": true,
  "count": 2,
  "products": [
    {
      "_id": "PRODUCT_ID",
      "name": "Coca Cola",
      "price": 250,
      "isActive": true
    }
  ]
}
```

4.3. Get Single Product by ID

Purpose: Returns one product using its MongoDB product ID.

Method	GET
--------	-----

Endpoint	/api/products/PRODUCT_ID
Full URL Example	http://localhost:5000/api/products/PRODUCT_ID

How to use in Postman: Copy product_id from Get All Products response and paste it in the URL.

Example Response:

```
{
  "success": true,
  "product": {
    "_id": "PRODUCT_ID",
    "name": "Coca Cola",
    "barcode": "123456789111",
    "price": 250
  }
}
```

4.4. Update Product with Image

Purpose: Updates product details. Image is optional. If a new image is selected, it uploads to Cloudinary and updates the MongoDB image URL.

Method	PUT
Endpoint	/api/products/PRODUCT_ID
Full URL Example	http://localhost:5000/api/products/PRODUCT_ID

How to use in Postman: Select Body -> form-data. Send only fields you want to update. Image field is optional.

Key	Type	Example Value
name	Text	Coca Cola Updated
barcode	Text	123456789111
brand	Text	Coca Cola
description	Text	Updated soft drink
price	Text	300
costPrice	Text	200
reorderLevel	Text	25
unit	Text	bottle
image	File	Choose new image - optional

Example Response:

```
{
  "success": true,
  "message": "Product updated successfully",
  "product": {
    "_id": "PRODUCT_ID",
    "name": "Coca Cola Updated",
    "price": 300,
    "image": "https://res.cloudinary.com/..."
  }
}
```

4.5. Deactivate Product

Purpose: Soft-deletes a product by changing isActive to false. The product is not removed from MongoDB.

Method	PATCH
Endpoint	/api/products/PRODUCT_ID/deactivate

Full URL Example	http://localhost:5000/api/products/PRODUCT_ID/deactivate
-------------------------	--

How to use in Postman: No body required. Use the product ID in the URL.

Example Response:

```
{
  "success": true,
  "message": "Product deactivated successfully",
  "product": {
    "_id": "PRODUCT_ID",
    "isActive": false
  }
}
```

4.6. Reactivate Product

Purpose: Activates an inactive product again by changing isActive to true.

Method	PATCH
Endpoint	/api/products/PRODUCT_ID/reactivate
Full URL Example	http://localhost:5000/api/products/PRODUCT_ID/reactivate

How to use in Postman: No body required. Use the product ID in the URL.

Example Response:

```
{
  "success": true,
  "message": "Product reactivated successfully",
  "product": {
    "_id": "PRODUCT_ID",
    "isActive": true
  }
}
```

4.7. Delete Product Permanently

Purpose: Permanently removes a product from MongoDB.

Method	DELETE
Endpoint	/api/products/PRODUCT_ID
Full URL Example	http://localhost:5000/api/products/PRODUCT_ID

How to use in Postman: No body required. Use carefully because this removes the product from the database.

Example Response:

```
{
  "success": true,
  "message": "Product deleted successfully"
}
```

4.8. Search Product by Barcode

Purpose: Finds a product by exact barcode. This is useful for POS billing and barcode scanner flow.

Method	GET
---------------	-----

Endpoint	/api/products/barcode/123456789111
Full URL Example	http://localhost:5000/api/products/barcode/123456789111

How to use in Postman: Replace the barcode value in the URL with the product barcode.

Example Response:

```
{
  "success": true,
  "product": {
    "_id": "PRODUCT_ID",
    "name": "Coca Cola",
    "barcode": "123456789111",
    "price": 250
  }
}
```

4.9. Search and Filter Products

Purpose: Searches products by name, barcode, brand, category, and price range. This is useful for the Product Management page.

Method	GET
Endpoint	/api/products/search/filter
Full URL Example	http://localhost:5000/api/products/search/filter

How to use in Postman: Use query parameters in the URL. Examples are given below.

Example Response:

```
{
  "success": true,
  "count": 1,
  "products": [
    {
      "_id": "PRODUCT_ID",
      "name": "Coca Cola",
      "brand": "Coca Cola",
      "price": 250
    }
  ]
}
```

Search URL examples:

```
GET http://localhost:5000/api/products/search/filter?keyword=coca
GET http://localhost:5000/api/products/search/filter?brand=Coca
GET http://localhost:5000/api/products/search/filter?minPrice=100&maxPrice=500
GET http://localhost:5000/api/products/search/filter?keyword=coca&minPrice=100&maxPrice=500
```

4.10. Get Active Products

Purpose: Returns only active products where isActive is true.

Method	GET
Endpoint	/api/products/status/active
Full URL Example	http://localhost:5000/api/products/status/active

How to use in Postman: No body required. Send a GET request.

Example Response:

```
{
  "success": true,
  "count": 1,
  "products": [
    {
      "_id": "PRODUCT_ID",
      "name": "Coca Cola",
      "isActive": true
    }
  ]
}
```

4.11. Get Inactive Products

Purpose: Returns only inactive products where isActive is false.

Method	GET
Endpoint	/api/products/status/inactive
Full URL Example	http://localhost:5000/api/products/status/inactive

How to use in Postman: No body required. Send a GET request.

Example Response:

```
{
  "success": true,
  "count": 1,
  "products": [
    {
      "_id": "PRODUCT_ID",
      "name": "Coca Cola",
      "isActive": false
    }
  ]
}
```

5. Endpoint Summary Table

Method	Endpoint	Purpose
POST	/api/products	Add product with image
GET	/api/products	Get all products
GET	/api/products/:id	Get single product
PUT	/api/products/:id	Update product with image
PATCH	/api/products/:id/deactivate	Deactivate product
PATCH	/api/products/:id/reactivate	Reactivate product
DELETE	/api/products/:id	Delete product permanently
GET	/api/products/barcode/:barcode	Find product by exact barcode
GET	/api/products/search/filter	Search/filter products
GET	/api/products/status/active	Get active products
GET	/api/products/status/inactive	Get inactive products

6. Important Notes

- For add and update with image, use form-data in Postman, not raw JSON.

- The image field name must be exactly image because the backend uses `upload.single("image")`.
- Barcode search is exact because barcode values should uniquely identify one product.
- Deactivate is safer than delete because it keeps the product in the database.
- Permanent delete should be used only when the product must be removed completely.
- Cloudinary stores the actual image. MongoDB stores only the image URL in the image field.